DarkWatch

Informe de avance · Semana 2 (Sprint 1: MVP funcional)

Interacción Hombre Máquina · Proyecto 3

Extensión de Chrome para detección de patrones oscuros con captura de DOM y motor de clasificación local

Grupo 4:	Fecha	Estado del sprint
Ariel Aguirre Edwin Toapanta Klever Lema	12 de abril de 2026	MVP funcional con flujo local operativo

Resumen. - Durante la semana 2 se implementó un MVP funcional de DarkWatch que ya captura el DOM de la página activa, ejecuta una detección heurística y utiliza un motor de clasificación local denominado “LLM demo local”. La integración remota quedó preparada mediante endpoint configurable, pero su conexión final se plantea para la siguiente iteración.

1. Contexto del proyecto y criterio de la semana 2

El Proyecto 3 pide desarrollar una extensión de navegador capaz de analizar una página web y detectar dark patterns. Para la semana 2 el foco es un MVP que capture el DOM, envíe información al motor de clasificación y muestre el panel de resultados con categorías y hallazgos.

PROYECTO 3: DarkWatch — Detector de Patrones Oscuros	
Dominio Ética en diseño / Dark Patterns / UX crítica	**Stack sugerido** JavaScript (Chrome Extension) + LLM API / Python + BeautifulSoup
Descripción Extensión de navegador o aplicación web que analiza una página y detecta automáticamente patrones oscuros (dark patterns) de diseño: urgencia falsa, confirmshaming, suscripciones ocultas, etc. Conecta la ética en diseño con la ingeniería de software y el análisis con IA.	
🚀 Semana 1 — Plan + Setup DevOps	▸ Investigar y catalogar 10 tipos de dark patterns ▸ Definir criterios de detección por tipo ▸ Configurar estructura de extensión Chrome (manifest v3) ▸ Crear repositorio con linting automático (ESLint)
⚛ Semana 2 — Sprint 1: MVP funcional	▸ Implementar captura de DOM de la página activa ▸ Enviar contenido al LLM para clasificación de dark patterns ▸ Mostrar panel con patrones detectados y su categoría ▸ Demo: analizar 5 e-commerce conocidos
🔍 Semana 3 — Sprint 2: Testing + Iteración	▸ Agregar severidad de cada patrón (Alto/Medio/Bajo) ▸ Captura de pantalla automática del elemento detectado ▸ Comparar resultados con catálogo ético (deceptive.design) ▸ Prueba con 5 sitios web distintos + validación de precisión
🎓 Semana 4 — Entrega Final + Demo	▸ Generar reporte exportable por sitio ▸ Publicar en Chrome Web Store (o build empaquetado) ▸ Presentación: demo en vivo + discusión ética de hallazgos ▸ Documentar falsos positivos y limitaciones
📦 ENTREGABLE FINAL: Extensión de Chrome instalable + base de datos de 20 sitios analizados + reporte ético comparativo	

Figura 1. Extracto del documento del módulo donde se define el Proyecto 3: DarkWatch y los objetivos del Sprint 1.

2. Objetivo del sprint y alcance implementado

El objetivo técnico de esta iteración fue construir un flujo mínimo pero demostrable que permita pasar de una página web real a un panel de análisis dentro de la extensión. El alcance implementado fue el siguiente:

- Captura del DOM de la página activa desde content.js.
- Extracción de texto visible, botones, modales, formularios, checkbox/radio y contadores tipo timer.
- Ejecución de reglas heurísticas para obtener hallazgos preliminares.
- Construcción de un prompt estructurado y clasificación con un motor local denominado “LLM demo local”.
- Panel visual en popup.js con resumen del análisis, riesgo, hallazgos y estadísticas del DOM.
- Preparación de un modo remoto con endpoint configurable para una integración futura.

2.1 Mapa de archivos principales

Archivo	Responsabilidad	Comentario relevante
manifest.json	Declara la extensión y sus scripts	Base de manifest v3 para la demo.
content.js	Captura el DOM y coordina el análisis	Obtiene snapshot y llama al clasificador.
rules/detectors.js	Detección heurística	Genera hallazgos preliminares por patrón.
services/llmClient.js	Motor local y cliente remoto	Aquí está localmente el “LLM demo local”.
popup.js	Interfaz del usuario	Permite seleccionar modo, analizar y visualizar resultados.
mock_server.py	Servidor de prueba para endpoint remoto	Se usa como apoyo para una futura integración.

3. LLM local dentro del proyecto

En esta versión no se utilizó una API real en producción. El componente llamado “LLM local” está implementado dentro de services/llmClient.js. Allí se concentran tres piezas clave: la construcción del prompt (buildPrompt), el clasificador local de demostración (runMockClassifier) y la función que decide si se usa el modo local o un endpoint remoto (classifyDarkPatternsWithLLM).

Esto significa que el análisis demostrado en semana 2 funciona de manera local dentro de la extensión. Técnicamente, el flujo visible es funcional, pero la parte remota se presenta como integración preparada y no como dependencia obligatoria del MVP.

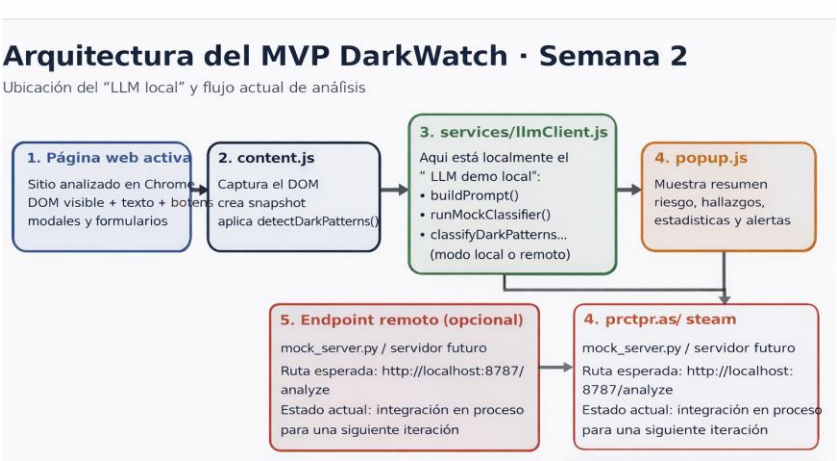

Figura 2. Ubicación del “LLM demo local” y flujo general del MVP implementado.

4. Sitios de Test

El propio resumen del proyecto sugiere mostrar la herramienta en 5 e-commerce conocidos. Esta sección debe incluirse antes de la conclusión, porque operacionaliza la demo pedida para la semana 2 y sirve como guía para la exposición en clase.

Sitio	Página / flujo sugerido	Qué observar	Patrón posible	Uso en la demo
Amazon	Página de producto	badges de oferta, urgencia visual, protección adicional	Interferencia visual / preselección	Sitio masivo y fácil de reconocer
eBay	Página de producto o puja	mensajes de disponibilidad o presión temporal	Urgencia falsa / escasez falsa	Muestra contraste entre compra y subasta
AliExpress	Producto con promoción	countdown, popups y mensajes de descuento	Urgencia falsa / misdirection	Muy útil para evaluar timers
Temu	Landing o página promocional	cupones, overlays, captación agresiva	Obstrucción / interferencia visual	Buena para mostrar popups y overlays
Shein	Carrito o checkout	costos, addons y pasos previos al pago	Costos ocultos / preselección	Permite discutir patrones cerca del pago

Recomendación práctica para la exposición: En cada sitio es preferible abrir una página de producto, carrito o promoción, porque allí suelen aparecer señales de urgencia, costos, preselecciones o interferencias visuales con mayor claridad.

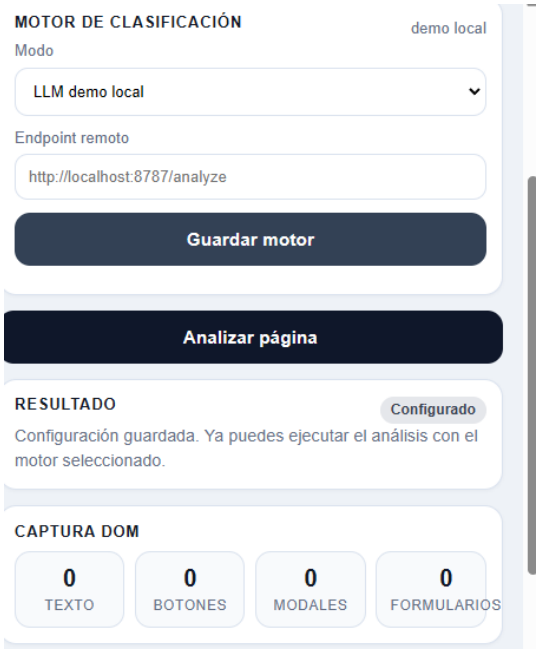

Figura 3. Interfaz del proyecto DarkWatch.

5. Funcionamiento actual del MVP

El comportamiento actual del sistema puede resumirse así:

- El usuario abre una página web y pulsa “Analizar página”.
- content.js captura el DOM y arma un snapshot con texto, botones, modales, formularios y timers.
- rules/detectors.js produce hallazgos heurísticos preliminares.
- services/llmClient.js construye un prompt y clasifica los hallazgos con el motor local.
- popup.js presenta el resumen, el nivel de riesgo, el conteo del DOM y los hallazgos detectados.

5.1 Pruebas

Su función principal en esta etapa es demostrar que el proyecto ya contempla un endpoint remoto configurable, aun cuando el flujo principal de la entrega sigue siendo el local.

Se ejecutó DarkWatch sobre una página de Amazon en español. La extensión logró capturar el DOM de la página activa y mostrar estadísticas de texto, botones, modales y formularios. En esta ejecución no se detectaron patrones oscuros con las reglas actualmente implementadas.

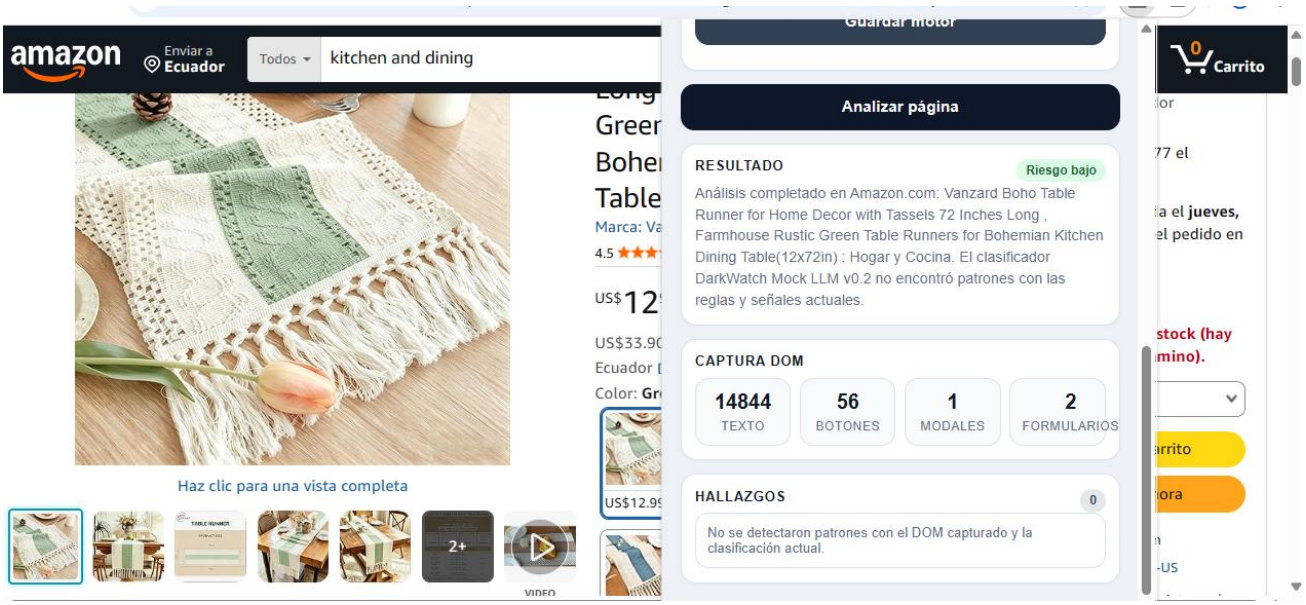

Figura 4. Prueba del MVP amazon.

Se analizó una página de eBay con resultados visibles en el panel de la extensión. La prueba confirmó el funcionamiento del flujo DOM → clasificación → visualización de resultados dentro del popup.

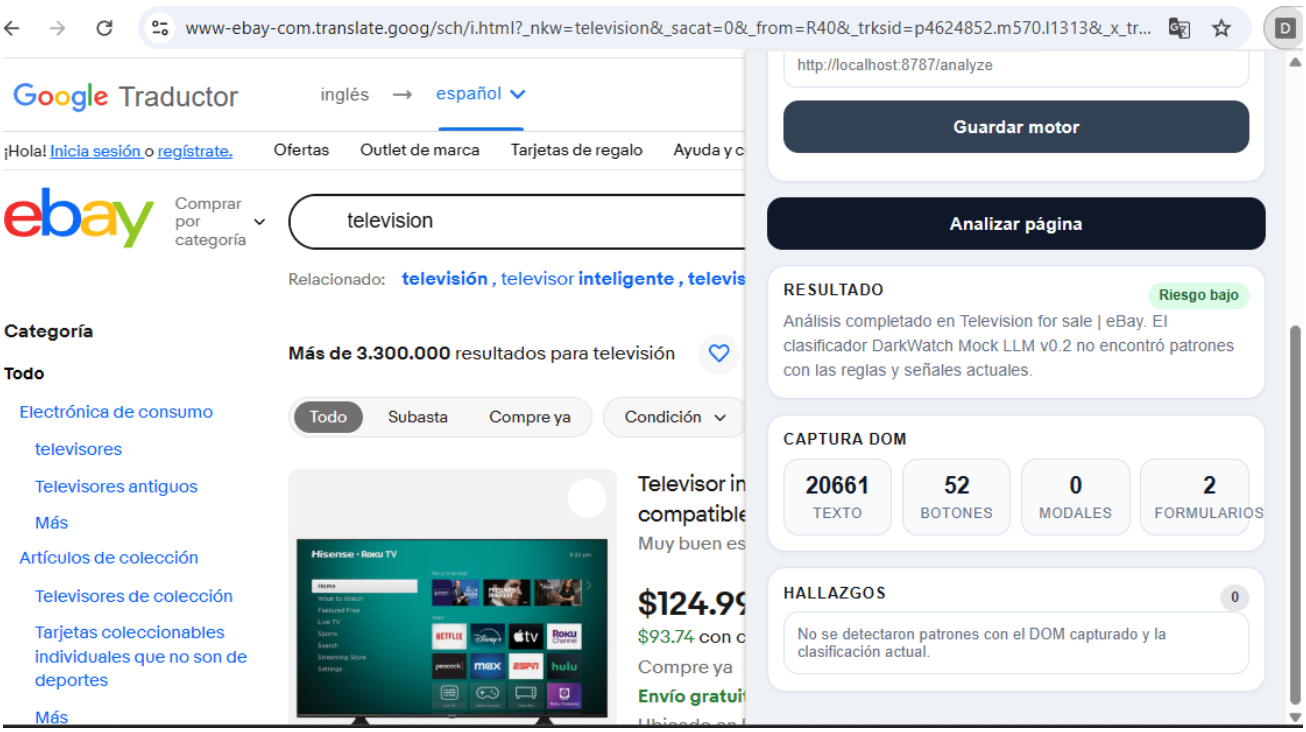

Figura 5. Prueba del MVP en eBay.

La extensión fue ejecutada sobre una página de producto/promoción en AliExpress. Se obtuvo un conteo del DOM y un resultado de bajo riesgo, sin hallazgos detectados por la versión actual del clasificador.

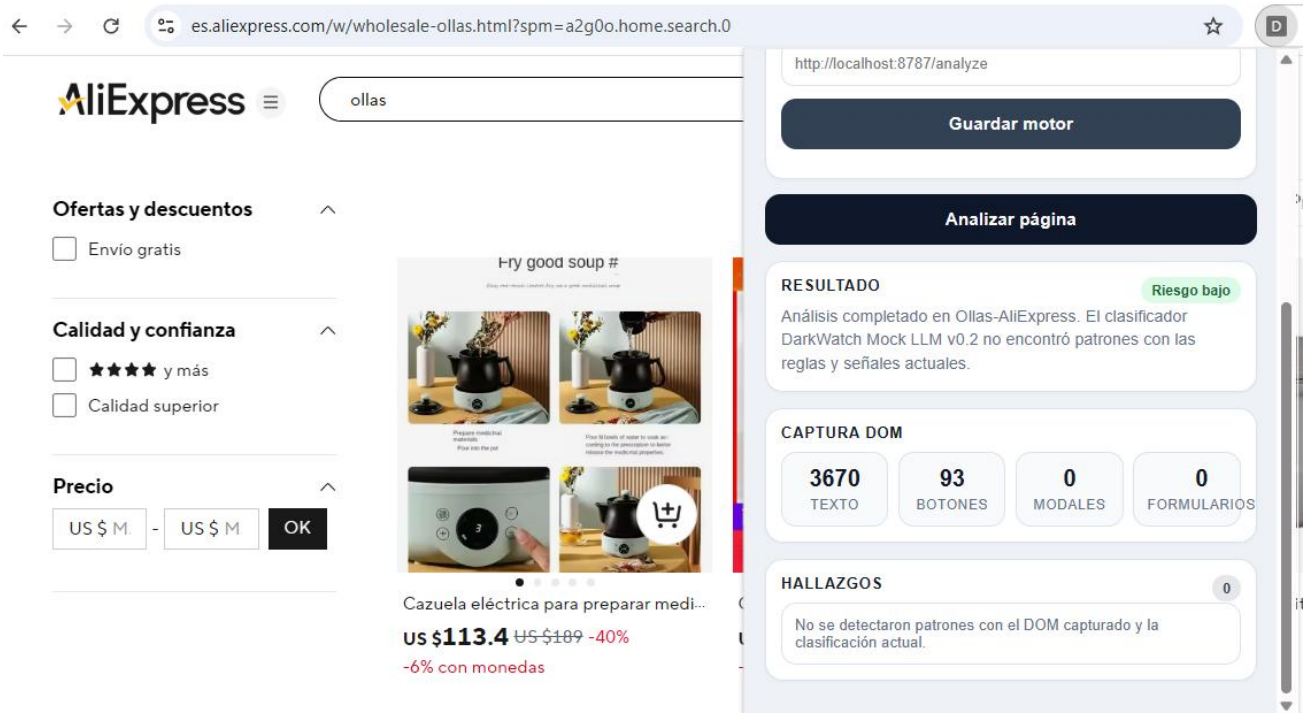

Figura 6. Prueba del MVP AliExpress

Se probó DarkWatch en Temu, una plataforma útil para analizar páginas con abundante contenido promocional. El sistema realizó la captura del DOM y completó el análisis local sin errores visibles en el flujo principal.

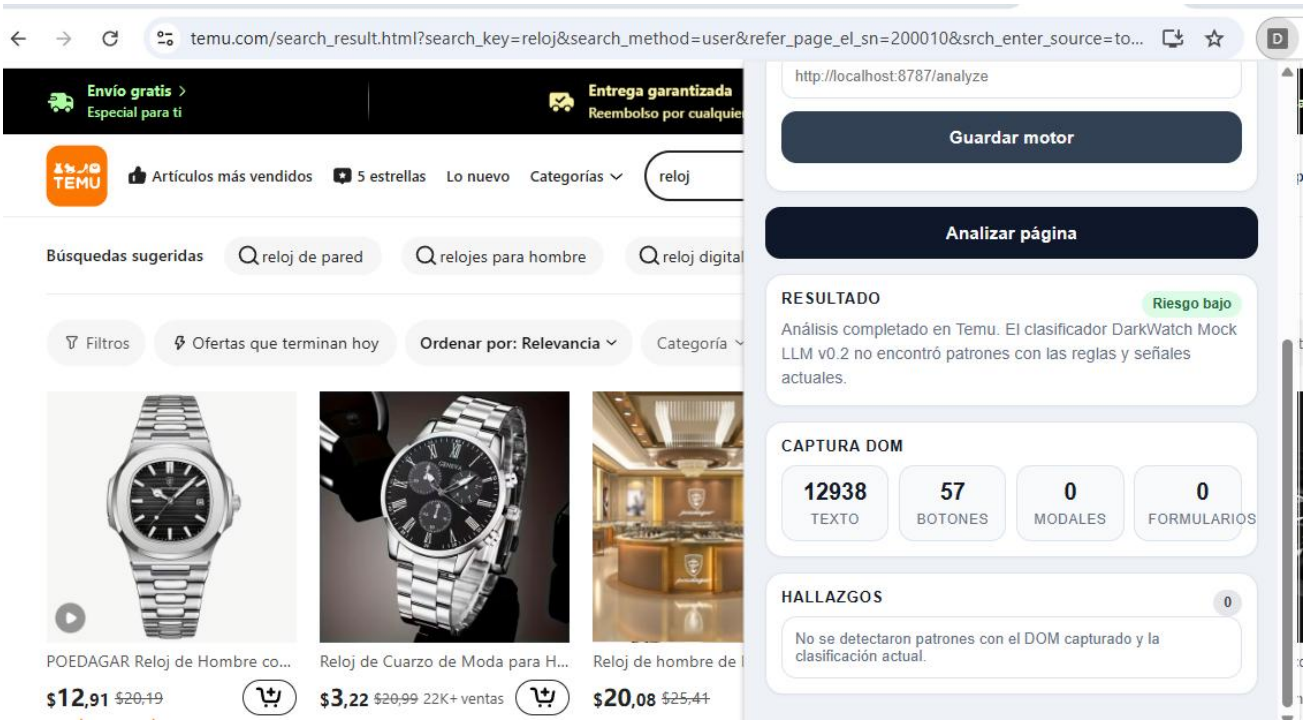

Figura 7. Prueba del MVP en Temu.

La prueba en Shein permitió validar el funcionamiento del prototipo sobre una interfaz visualmente cargada y orientada a comercio electrónico. En esta iteración, la extensión no detectó patrones con las reglas y señales actualmente definidas.

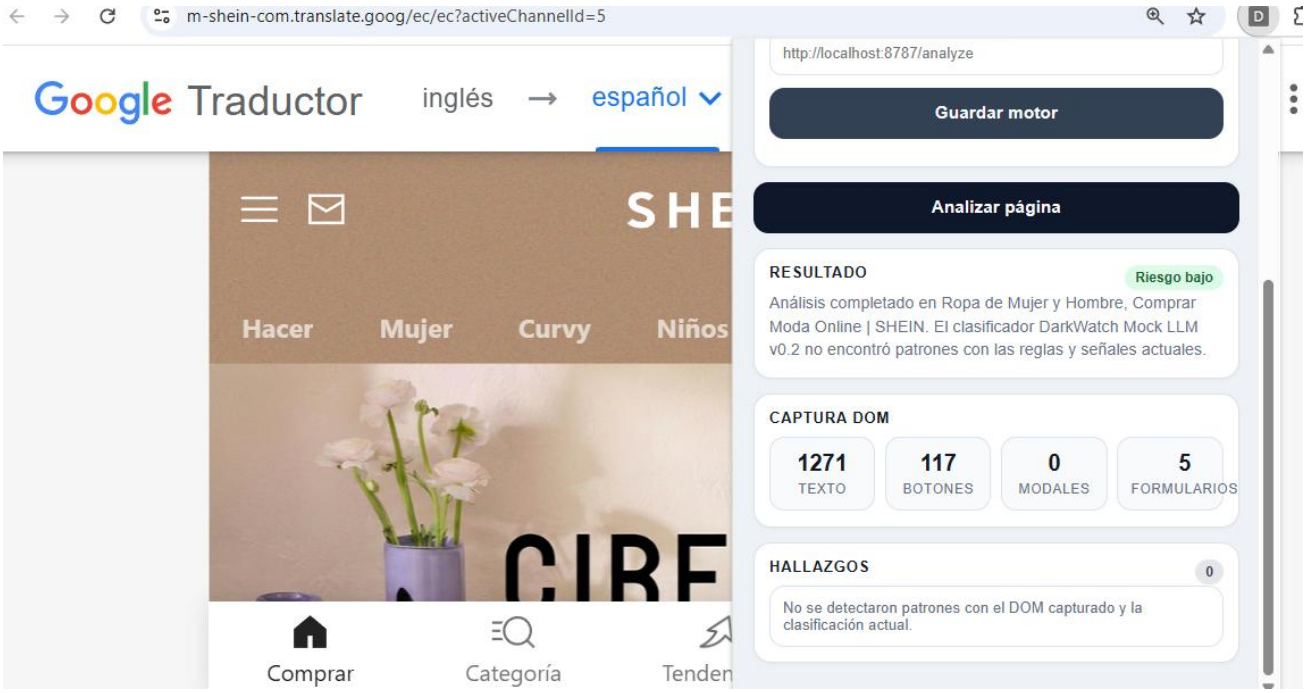

Figura 8. Prueba del MVP en shein.

En las cinco pruebas realizadas, DarkWatch logró completar el flujo de análisis sin afectar la navegación del usuario. Aunque en esta iteración no se detectaron patrones oscuros mediante las reglas actuales, las capturas demuestran que la extensión ya es funcional, obtiene métricas del DOM y está preparada para mejorar la precisión en la siguiente fase del proyecto.

6. Problemas encontrados y decisiones adoptadas

Durante el sprint aparecieron dos limitaciones principales. La primera fue un error de red al intentar ejecutar npm install, relacionado con timeout y conectividad del registro de paquetes.

```
PS D:\PROJECT\Master\Software\Cursos\HombreMaquina\Proyecto\darkwatch> npm install
npm error network If you are behind a proxy, please make sure that the 'proxy' config is set p
roperly.  See: 'npm help config'
npm error A complete log of this run can be found in: C:\Users\Klever\AppData\Local\npm-cache\
_logs\2026-04-12T19_26_37_006Z-debug-0.log
PS D:\PROJECT\Master\Software\Cursos\HombreMaquina\Proyecto\darkwatch>
```

Figura 9. Problemas de descarga del nmp install.

La segunda fue que la integración remota aún no quedó totalmente estabilizada dentro del flujo final de la extensión.

Problema técnico	Decisión del sprint
• Timeout en npm install • Endpoint remoto todavía no consolidado • Riesgo de perder tiempo del sprint depurando infraestructura	• Priorizar el flujo local funcional • Mantener el modo remoto como integración preparada • Presentar el MVP de forma honesta y defendible

Por esta razón, para la demostración de semana 2 se recomienda presentar el motor local como componente estable del MVP y describir el endpoint remoto como “integración en proceso”. Esta decisión es coherente con el criterio del sprint, porque lo realmente evaluable en esta fase es que el sistema ya recorra el flujo DOM → clasificación → panel visible.

7. Conclusión

El avance de la semana 2 puede considerarse satisfactorio como MVP funcional. DarkWatch ya captura el DOM de la página activa, produce estadísticas relevantes, ejecuta una clasificación local y muestra un panel de resultados utilizable. Además, la arquitectura ya deja visible dónde se encuentra el motor local y cómo se conectaría un futuro endpoint remoto.

Para la siguiente iteración se recomienda concentrarse en tres frentes: afinar la integración remota, mejorar la precisión de los hallazgos y capturar evidencia visual automática del elemento detectado. Con eso, el proyecto quedará mejor posicionado para la semana 3 y para la demostración final.